Bachelor of Computer Applications (Hons. AI and ML)
Semester- III

**Subject: Database Management System**

# Assignment 8: Basic SQL Queries and Aggregate Functions

Case Study: Online Food Delivery System

By
**Name:** AKASH P
**Reg. No.:** 2511021240008

**Faculty Name: Renita Joy**
**Faculty Signature:_____**

**Department of Computer Application**
**Alliance University**
**Chandapura - Anekal Main Road, Anekal**
**Bengaluru - 562 106**
**October 2026**

# 1. Introduction

These queries run against a sample Online Food Delivery System dataset (modeled on platforms such as Swiggy or Zomato). Registered customers browse restaurants, view menus, place orders, pay online and receive the food through a delivery agent, after which they can submit a rating. The database stores customers, restaurants, menu items, delivery agents, orders, payments and reviews.

**Tables used:** Customers, Restaurants, MenuItems, DeliveryAgents, Orders, Payments, Reviews. The sample data has 6 customers, 5 restaurants, 10 menu items, 3 delivery agents, 8 orders, 8 payments and 6 reviews.

## 1. Filter active customers

Lists customers whose account status is Active, ordered by customer ID.

```
SELECT customer_id, name, city, account_status
FROM Customers
WHERE account_status = 'Active'
ORDER BY customer_id;
```


```
SELECT customer_id, name, city, account_status
FROM Customers
WHERE account_status = 'Active'
ORDER BY customer_id;
```


**RESULT GRID (illustrative output from the sample dataset)**

| customer_id | name | city | account_status |
|---|---|---|---|
| 1 | Rahul Verma | Bengaluru | Active |
| 2 | Sneha Reddy | Bengaluru | Active |
| 4 | Pooja Nair | Bengaluru | Active |
| 6 | Divya Menon | Bengaluru | Active |

## 2. Sort menu items by price

Displays menu items from the most expensive to the cheapest.

```
SELECT item_name, price
FROM MenuItems
ORDER BY price DESC;
```

```
SELECT item_name, price
FROM MenuItems
ORDER BY price DESC;
```

**RESULT GRID (illustrative output from the sample dataset)**

| item_name | price |
|---|---|
| Farmhouse Pizza | 350.0 |
| Butter Chicken | 320.0 |
| Margherita Pizza | 280.0 |
| Paneer Tikka | 260.0 |
| Chilli Chicken | 240.0 |
| Veg Hakka Noodles | 180.0 |
| Garlic Bread | 120.0 |
| Masala Dosa | 90.0 |
| Idli Vada | 70.0 |
| Garlic Naan | 60.0 |

## 3. Count menu items in each restaurant (GROUP BY)

Counts menu items per restaurant, including restaurants with zero items.

```
SELECT r.restaurant_name, COUNT(m.item_id) AS item_count
FROM Restaurants r
LEFT JOIN MenuItems m ON m.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_id, r.restaurant_name
ORDER BY item_count DESC, r.restaurant_name;
```

```
SELECT r.restaurant_name, COUNT(m.item_id) AS item_count
FROM Restaurants r
LEFT JOIN MenuItems m ON m.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_id, r.restaurant_name
ORDER BY item_count DESC, r.restaurant_name;
```

**RESULT GRID (illustrative output from the sample dataset)**

| restaurant_name | item_count |
|---|---|
| Pizza Hub | 3 |
| Spice Garden | 3 |
| Dosa Corner | 2 |
| Wok Express | 2 |
| Sweet Tooth | 0 |

## 4. Restaurants with at least three menu items (HAVING)

HAVING filters grouped results after aggregation.

```
SELECT r.restaurant_name, COUNT(m.item_id) AS item_count
FROM Restaurants r
JOIN MenuItems m ON m.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_id, r.restaurant_name
HAVING COUNT(m.item_id) >= 3
ORDER BY item_count DESC, r.restaurant_name;
```


```
SELECT r.restaurant_name, COUNT(m.item_id) AS item_count
FROM Restaurants r
JOIN MenuItems m ON m.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_id, r.restaurant_name
HAVING COUNT(m.item_id) >= 3
ORDER BY item_count DESC, r.restaurant_name;
```


**RESULT GRID (illustrative output from the sample dataset)**

| restaurant_name | item_count |
|---|---|
| Pizza Hub | 3 |
| Spice Garden | 3 |

## 5. Aggregate functions

Demonstrates COUNT, SUM, AVG, MIN and MAX on the eight sample payment records.

```
SELECT
  COUNT(*) AS total_payments,
  SUM(amount) AS total_amount,
  ROUND(AVG(amount), 2) AS avg_amount,
  MIN(amount) AS smallest_payment,
  MAX(amount) AS largest_payment
FROM Payments;
```


```
SELECT
  COUNT(*) AS total_payments,
  SUM(amount) AS total_amount,
  ROUND(AVG(amount), 2) AS avg_amount,
  MIN(amount) AS smallest_payment,
  MAX(amount) AS largest_payment
FROM Payments;
```


**RESULT GRID (illustrative output from the sample dataset)**

| total_payments | total_amount | avg_amount | smallest_payment | largest_payment |
|---|---|---|---|---|
| 8 | 3060.0 | 382.5 | 160.0 | 640.0 |

### 6. Average rating per restaurant

Shows the number of reviews and the average customer rating for each restaurant.

```
SELECT r.restaurant_name, COUNT(rv.review_id) AS total_reviews,
       ROUND(AVG(rv.rating), 2) AS avg_rating
FROM Reviews rv
JOIN Orders o ON o.order_id = rv.order_id
JOIN Restaurants r ON r.restaurant_id = o.restaurant_id
GROUP BY r.restaurant_id, r.restaurant_name
ORDER BY avg_rating DESC, r.restaurant_name;
```


```
SELECT r.restaurant_name, COUNT(rv.review_id) AS total_reviews,
       ROUND(AVG(rv.rating), 2) AS avg_rating
FROM Reviews rv
JOIN Orders o ON o.order_id = rv.order_id
JOIN Restaurants r ON r.restaurant_id = o.restaurant_id
GROUP BY r.restaurant_id, r.restaurant_name
ORDER BY avg_rating DESC, r.restaurant_name;
```


**RESULT GRID (illustrative output from the sample dataset)**

| restaurant_name | total_reviews | avg_rating |
|---|---|---|
| Spice Garden | 1 | 5.0 |
| Dosa Corner | 2 | 4.0 |
| Pizza Hub | 2 | 4.0 |
| Wok Express | 1 | 4.0 |

# 2. Summary

WHERE filters individual rows, ORDER BY controls display order, GROUP BY forms groups for aggregation, and HAVING filters those groups. COUNT, SUM, AVG, MIN and MAX summarize values across the selected records.